AGENT BEHAVIORAL INTELLIGENCE PLATFORM

Deterministic observability for autonomous coding agents

Kunal Anand

Bharat Academix CodeQuest

Round 1 :- **Idea Submission**

THE PROBLEM

AI coding agents are becoming unpredictably expensive — and nobody can explain why.

1,000×

more tokens used by agentic tasks vs. ordinary code chat

Stanford Digital Economy Lab, 2026

30×

variance in token cost for the identical task, same output quality

Stanford Digital Economy Lab, 2026

$500M

spent by one enterprise on Claude in a single month — no usage caps

industry reporting, 2026

Who is affected

- Developers hit with unpredictable, unexplained session costs
- Security teams blind to agent access of credential files
- Teams running agents in CI/CD with no usable post-mortem
- Engineering leaders needing auditable AI usage evidence

The gap: Existing tools (LangSmith, Helicone, OpenTelemetry) show cost dashboards and raw traces — they tell you that money was spent, never *why*. Context optimizers reduce future usage but don't diagnose past behavior. None of them reason about what the agent actually did.

WHY THIS IS DIFFERENT

Every AI observability tool today tells you ***what*** an agent did. Ours is the first to deterministically tell you ***why*** it cost what it cost — without using an LLM to explain an LLM.

Deterministic, not an LLM wrapper

Every core finding is rule-based and reproducible. Same session, same diagnosis — no hallucinated explanations.

Cross-agent normalization

No public tool unifies Claude Code, OpenHands, SWE-agent, and LangSmith traces into one schema.

Diagnosis, not dashboards

Pinpoints the exact retry loop or redundant read causing cost — not just a cost total.

Security visibility, first-class

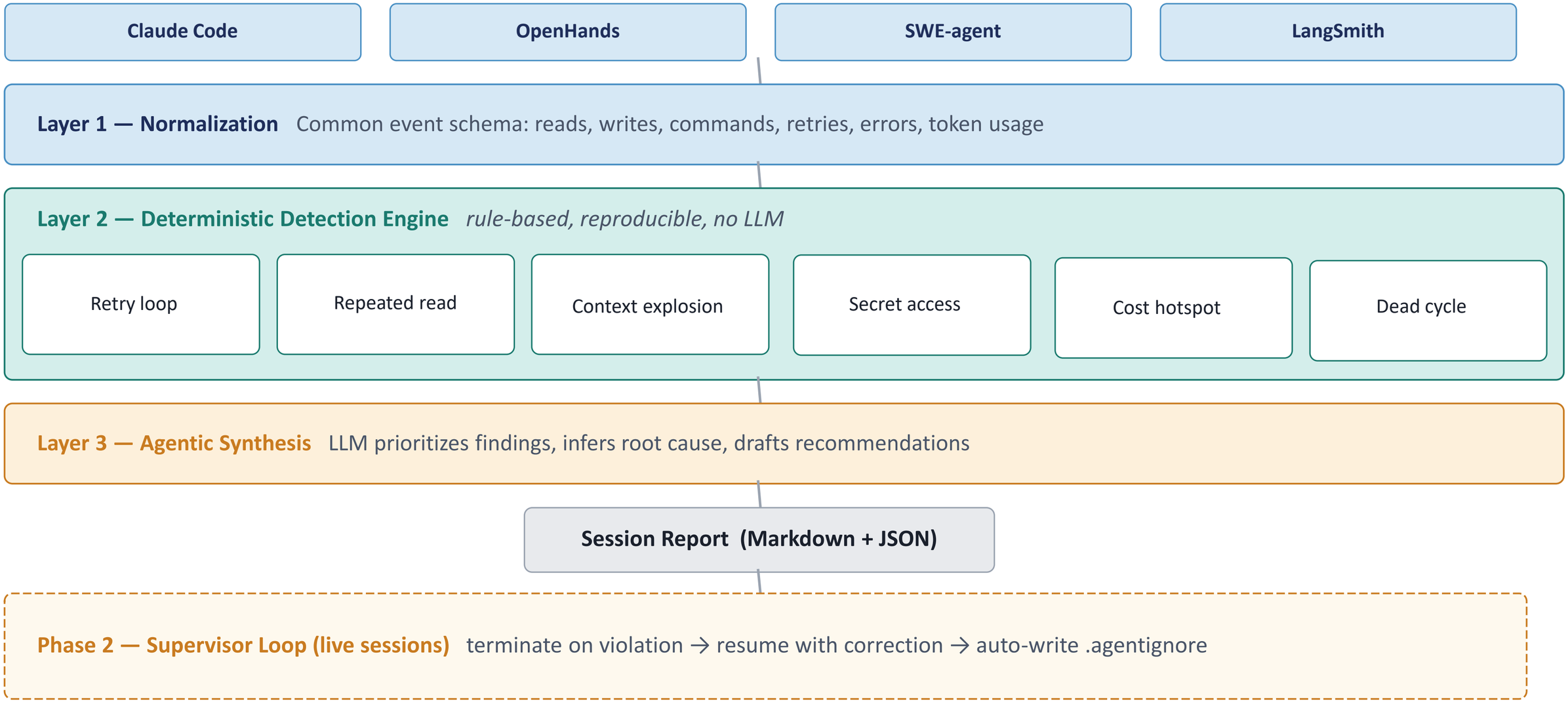
No competing platform treats secret-file access during a session as a tracked behavioral signal.

Closes the loop

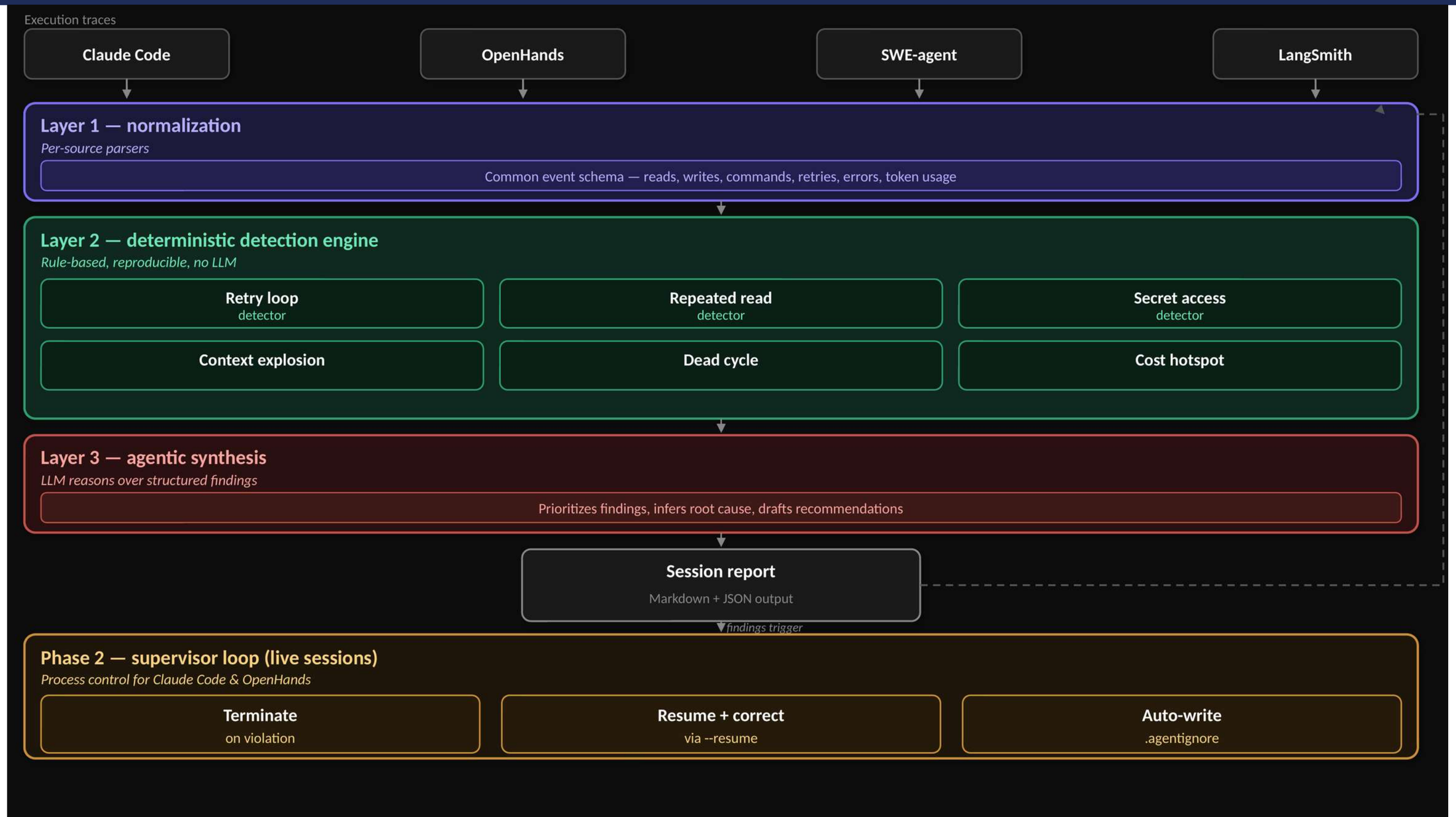
Detection feeds remediation — auto .agentignore rules, regenerated prompts, and live intervention.

SOLUTION ARCHITECTURE

Three deterministic layers, plus an optional live supervision loop



FLOW DIAGRAM



PER-AGENT CAPABILITY MATRIX

Honest, verified support — not uniform claims across all agents

Capability	Claude Code	OpenHands	SWE-agent
Post-session log analysis	Yes	Yes	Yes
Live streaming detection	Yes (stream-json)	Yes (--json headless)	No — batch only
Process termination on violation	Yes	Yes	Yes
Resume with corrective instruction	Yes (--resume)	Yes (--resume <id>)	No resume primitive
Auto-remediation (.agentignore, prompt rewrite)	Yes	Yes	Yes

Why SWE-agent differs: Its batch-oriented design returns a complete .traj file after the run finishes — there is no documented streaming or resume primitive comparable to Claude Code or OpenHands. This is a limitation of the target tool's architecture, not a gap in our engineering. Live supervision for SWE-agent-style tools is listed as future work, pending a streaming interface from the tool itself.

SAMPLE OUTPUT

A real example finding — not an aspirational claim

```
SESSION REPORT — fastapi-backend / sess-0001

⚠  RETRY LOOP DETECTED

   Command: pytest tests/test_auth.py — repeated 5x, identical ImportError each time

   Time burned: 38s   |   Tokens burned: ~9,200 (~$0.03)

   Fix: Agent never modified the import target between retries — stop after 2 identical failures.

🔒 SECRET FILE ACCESS

   Files read: .env, .env.backup, config/production.py

   Contains: DB credentials, AWS keys, Stripe live key

   Fix: Add to .agentignore: .env*, config/production.py

✓  EFFICIENT PATH: utils/tokens.py write → test run → pass

   Benchmark: session sess-0005, same task type, 71% fewer tokens
```

Output formats: Markdown (human-readable) · JSON (machine-readable, CI-integrable)

TECHNOLOGY STACK

Core Engine

- Python 3.12+
- Pydantic — schema validation
- Typer — CLI interface
- jsonlines / ijson (Crucial for streaming huge jsonl trace logs without blowing up RAM)

Ingestion

- Custom parsers per source format
- watchdog — live file watching

Detection Layer

- Pure Python rule-based detectors
- Fully unit-testable, zero external deps

Synthesis (Phase 1.5)

- Anthropic API (Claude Sonnet)
- Single-call structured summarization

Supervisor (Phase 2)

- Python subprocess management
- Native agent CLI primitives (--resume)

Backend & Deploy

- FastAPI — REST API & Uvicorn Server
- PostgreSQL & SQLAlchemy (Async Session Storage)
- Docker — CI pipeline packaging

EXPECTED IMPACT

If even 15–20% of the documented 30× token variance is recoverable waste, a team spending $400K/year on agentic tools could realistically recover **$60,000–$80,000/year** in avoidable token spend — with no change to the underlying model.

Estimate based on verified research variance — not a guaranteed figure

Cost Control

Itemized, avoidable token spend per session — actionable, not just alarming

Faster Debugging

Structured failure reports instead of raw log archaeology after a failed run

Security Visibility

Full audit trail for every sensitive file accessed during an agent session

Governance Evidence

Concrete data to justify or course-correct AI tooling spend to leadership

Ecosystem Tooling

Cross-format normalizer is independently valuable to researchers and OSS projects

PROJECT ROADMAP

Phase 1

Deterministic Core

Buildable Now — MVP

- Normalizer for Claude Code, OpenHands, SWE-agent
- Core detectors: retry loops, repeated reads, secret access, cost hotspots
- CLI: agent-analyzer session.jsonl --format markdown

Phase 1.5

Agentic Synthesis

Near-Term Extension

- LLM root-cause summarization over detector findings
- Auto-remediation: .agentignore generation, prompt rewrite suggestions

Phase 2

Supervisor Loop

Stretch Goal — Live Demo

- Subprocess management for Claude Code & OpenHands
- Real-time detection + checkpoint-and-correct on violation
- LangSmith integration for LangChain-based agents

Phase 3

Future Work

Explicitly Out of Scope

- Network-level exfiltration monitoring (sandbox/OS scope)
- Live credential revocation via provider APIs
- Web dashboard with historical trend analysis